TRANSMISIÓN DE UNA IMAGEN UTILIZANDO USRPs Y SIMULINK/MATLAB

Documentación Sprint 2 — Preparación del flujo de datos

Laboratorio II de Telecomunicaciones

Universidad del Cauca — Facultad de Ingeniería Electrónica y Telecomunicaciones

Tabla 1. Integrantes del Equipo de desarrollo.

Integrante	Rol
Luis Miguel Gomez Muñoz	Desarrollador
Jose Martin Gonzalez Joaqui	Desarrollador
Briyith Vaneza Guacas Narvaez	Desarrolladora
Karol Tatiana Palechor Valencia	Desarrolladora

1. Descripción del Sprint 2.

El Sprint 2 tuvo como objetivo preparar el flujo de datos y la trama lista para transmisión. Se partió de una imagen a color en formato JPG, se procesó en MATLAB mediante el script *main.m* (integración de las funcionalidades descritas posteriormente) y se exportaron las variables resultantes al workspace para su posterior uso en Simulink a través del bloque To Workspace.

Fecha de inicio: 28 de febrero de 2026.

Fecha de finalización: 6 de marzo de 2026.

2. Distribución de Tareas.

Tabla 2. Distribución de Tareas de Sprint 2.

Tarea	Inicio	Fin	Responsable
Cargar la imagen en MATLAB/Simulink y obtener la matriz de píxeles	28 Feb	28 Feb	Jose Martin Gonzalez Joaqui
Aplicar compresión por paleta para obtener imagen indexada de hasta 256 colores y guardar la paleta	1 Mar	2 Mar	Karol Tatiana Palechor Valencia
Convertir la matriz indexada en una secuencia unidimensional de píxeles indexados	2 Mar	2 Mar	Briyith Vaneza Guacas Narvaez
Convertir la secuencia de píxeles indexados a una secuencia binaria e integrar el flujo de procesamiento del transmisor	3 Mar	6 Mar	Luis Miguel Gomez Muñoz

3. Implementación.

Toda la lógica del Sprint 2 fue implementada en el archivo *cargar_imagen.m*. Este script realiza de forma secuencial las cuatro etapas del sprint y exporta los resultados al workspace de MATLAB para su uso en Simulink mediante el bloque To Workspace del modelo *transmisor.slx.*

3.1. Carga de imagen y obtención de la matriz de píxeles.

Responsable: Jose Martin Gonzalez Joaqui

Se cargó la imagen *italy.jpg* usando la función *imread()* de MATLAB. La imagen es de tipo RGB con tres canales (rojo, verde, azul). Se verificó que cumpliera el tamaño mínimo de 740×492 píxeles; en caso de no cumplirlo, se redimensionó automáticamente usando imresize().

Código implementado:

```
None
function [imagen, filas, columnas, canales] = cargar_imagen(nombre_archivo)
% CARGAR_IMAGEN Carga una imagen y verifica/redimensiona al mínimo 740x492
% Entrada:  nombre_archivo -> nombre del archivo
% Salidas:  imagen, filas, columnas, canales

    % 1. Cargar la imagen
    imagen = imread(nombre_archivo);

    % 2. Verificar y redimensionar si es necesario
    [filas, columnas, canales] = size(imagen);

    if columnas < 740 || filas < 492
        imagen = imresize(imagen, [492, 740]);
        [filas, columnas, canales] = size(imagen);
    end

end
```

Resultado obtenido:

La imagen original tiene dimensiones 1021×828×3, las cuales superan el mínimo requerido. Las variables imagen, *matriz_R, matriz_G y matriz_B* y la imagen original quedaron disponibles en el workspace de MATLAB.

3.2. Compresión por paleta e imagen indexada.

Responsable: Karol Tatiana Palechor Valencia

La elaboración de esta tarea hace que las matrices RGB creadas en la tarea anterior no sean de utilidad, debido a que la función para aplicar la compresión por paleta requería de la imagen original completa. Por lo tanto, se usó la imagen completa obtenida de la tarea anterior como entrada de la

función. Se aplicó compresión por paleta usando la función *rgb2ind()* de MATLAB con un máximo de 256 colores. Esta técnica reduce la representación de cada píxel de 24 bits (3 canales × 8 bits) a 8 bits (índice en la paleta), disminuyendo la cantidad de datos a transmitir. La paleta de colores generada se guardó en el archivo *paleta_colores.mat* para su uso posterior en la reconstrucción de la imagen en el receptor.

Código implementado:

```
None
function [imagen_indexada, paleta] = comprimir_paleta(imagen)
% COMPRIMIR_PALETA Aplica compresión por paleta indexada de 256 colores
% Entrada:  imagen         -> imagen RGB uint8
% Salidas:  imagen_indexada -> imagen comprimida con índices
%           paleta          -> paleta de 256 colores [256x3]

    % Aplicar compresión por paleta
    [imagen_indexada, paleta] = rgb2ind(imagen, 256);

    % Guardar paleta para uso en receptor
    save('paleta_colores.mat', 'paleta');

end
```

Resultado obtenido:

Se generó la variable *imagen_indexada* de tipo uint8 con dimensiones 1021×828 y una paleta de 256 colores representada como una matriz de 256×3 valores de tipo double en el rango [0, 1]. El archivo *paleta_colores.mat* fue guardado en la carpeta *desarrollo/martin* del repositorio.

3.3. Serialización: matriz indexada a secuencia unidimensional.

Responsable: Briyith Vaneza Guacas Narvaez

Se convirtió la matriz bidimensional imagen_indexada en un vector unidimensional usando el operador (:) de MATLAB. Este proceso recorre la matriz columna por columna y genera una secuencia lineal de índices de píxeles, lo que permite su procesamiento posterior como un flujo de datos serial. Posteriormente, se realiza la comprobación de la longitud del vector columna obtenido. Si tiene el tamaño esperado, imprime en un mensaje, y si no tiene el tamaño esperado, imprime en otro mensaje.

Código implementado:

```
None
function secuencia_pixeles = serializar_pixeles(imagen_indexada, filas,
columnas)
% SERIALIZAR_PIXELES Convierte la matriz indexada en secuencia
unidimensional
```

```
% Entrada:  imagen_indexada  -> matriz 2D de índices de píxeles
%           filas, columnas  -> dimensiones esperadas
% Salida:   secuencia_pixeles -> vector 1D de índices

    % Serializar la matriz en un vector
    secuencia_pixeles = imagen_indexada(:);

    % Verificar longitud esperada
    if length(secuencia_pixeles) ~= (filas * columnas)
        longitud_real = filas * columnas;
        secuencia_pixeles = secuencia_pixeles(0:longitud_real);
    end

end
```

Resultado obtenido:

Se obtuvo el vector *secuencia_pixeles* de longitud 845.388 elementos (1021×828), verificando que coincide con el número total de píxeles de la imagen.

3.4. Conversión de índices de píxeles a secuencia binaria.

Responsable: Luis Miguel Gomez Muñoz

Cada índice de la secuencia de píxeles fue convertido a su representación binaria de 8 bits usando la función de2bi() con orden de bits MSB primero (bit más significativo a la izquierda). La matriz binaria resultante fue transpuesta y serializada en un vector unidimensional que representa el flujo de bits listo para ser procesado por el codificador convolucional y el modulador QPSK en los siguientes sprints. Posteriormente, se realiza la verificación del tamaño esperado del vector obtenido. Si tiene el tamaño esperado, imprime en un mensaje, y si no tiene el tamaño esperado, imprime en otro mensaje.

Código implementado:

```
None
function secuencia_binaria = convertir_binario(secuencia_pixeles, filas,
columnas)
% CONVERTIR_BINARIO Convierte la secuencia de píxeles indexados a bits
% Entrada:  secuencia_pixeles -> vector 1D de índices uint8
%           filas, columnas   -> dimensiones de la imagen
% Salida:   secuencia_binaria -> vector 1D de bits (0s y 1s)

    % Convertir cada índice a 8 bits
    binario_pixeles = de2bi(secuencia_pixeles, 8, 'left-msb');

    % Serializar la matriz binaria en un vector
    secuencia_binaria = binario_pixeles';
```

```
    secuencia_binaria = secuencia_binaria(:);

    % Verificar tamaño
    num_bits = length(secuencia_binaria);
    if num_bits ~= (filas * columnas * 8)
        longitud_real = filas * columnas * 8;
        secuencia_binaria = secuencia_binaria(0:longitud_real);
    end

end
```

Resultado obtenido:

Se generó el vector secuencia_binaria con 6.763.104 bits (845.388 píxeles × 8 bits), verificando que el tamaño es correcto.

3.5. Integración del flujo de procesamiento del transmisor

Responsable: Luis Miguel Gomez Muñoz

En esta etapa se realizó la integración de las funciones desarrolladas en los módulos anteriores, consolidándose dentro de un archivo principal que permite ejecutar de manera secuencial todo el flujo de procesamiento del transmisor.

El archivo principal coordina las siguientes etapas del sistema:

1. Carga de la imagen: se lee el archivo de imagen desde el sistema y se obtienen sus dimensiones.

2. Compresión mediante paleta de colores: la imagen RGB es convertida a una representación indexada utilizando una paleta, reduciendo la información necesaria para representar cada píxel.

3. Serialización de píxeles: los índices de píxeles de la imagen se reorganizan en un vector unidimensional que representa la secuencia de píxeles a transmitir.

4. Conversión a secuencia binaria: cada índice de píxel es transformado a su representación binaria de 8 bits y posteriormente serializado en un flujo de bits.

De esta manera, el archivo principal permite encadenar todos los módulos del sistema, generando como salida final la secuencia de bits que será utilizada en los siguientes sprints para los procesos de codificación convolucional y modulación QPSK.

Código implementado:

```
None
% MAIN - Archivo principal del transmisor Sprint 2
```

```
% ── Función 1 Cargar imagen ──────────────────────────────
[imagen, filas, columnas, canales] = cargar_imagen('italy.jpg');

% ── Función 2 Compresión por paleta ──────────────────────
[imagen_indexada, paleta] = comprimir_paleta(imagen);

% ── Función 3 Serialización de píxeles ───────────────────
secuencia_pixeles = serializar_pixeles(imagen_indexada, filas, columnas);

% ── Función 4 Conversión a binario ───────────────────────
secuencia_binaria = convertir_binario(secuencia_pixeles, filas, columnas);
```

Resultado obtenido:

Se logró integrar correctamente los módulos desarrollados en los sprints anteriores dentro de un único flujo de ejecución. El sistema permite procesar una imagen de entrada, convertirla a una representación indexada, serializar sus píxeles y generar la secuencia binaria correspondiente, la cual será utilizada como entrada para los bloques de codificación y modulación del sistema de transmisión digital en las siguientes etapas del proyecto.

4. Integración con Simulink.

Las variables generadas por el script *main.m* son exportadas automáticamente al Workspace de MATLAB usando la función *assignin()*. El modelo *transmisor.slx* accede a estas variables mediante el bloque To Workspace de Simulink, permitiendo que el flujo de procesamiento continúe en el entorno de simulación gráfica para las etapas siguientes del sistema.

Las variables disponibles en el workspace al finalizar la ejecución del script son:

Tabla 3. Variables de la ejecución del script *main.m*

Variable	Dimensiones	Descripción
imagen	1021×828×3 uint8	Imagen original RGB cargada
imagen_indexada	1021×828 uint8	Imagen comprimida con paleta de 256 colores
paleta	256×3 double	Paleta de colores para reconstrucción
secuencia_pixeles	845.388×1 uint8	Secuencia unidimensional de índices
secuencia_binaria	6.763.104×1 double	Flujo de bits listo para codificación

5. Cambios realizados.

Las actividades y entregables fueron pensadas para el desarrollo en un archivo SLX debido a que se tiene pensado trabajar con Simulink, sin embargo; durante el desarrollo se observó que durante la Actividad 2: Aplicar compresión por paleta para obtener una imagen indexada de hasta 256 colores y guardar la paleta era necesario realizarlo mediante un script de MATLAB (nombrado como *main.m*). De esta forma, el procesamiento del archivo se genera en un archivo llamado *paleta_colores.mat* que se importará al archivo *transmisor.slx* mediante el bloque To Workspace.

6. Entregables del Sprint 2.

Tabla 4. Entregables de Sprint 2.

Entregable	Estado
Imagen cargada	Completado
Técnica de compresión aplicada	Completado
Serialización de píxeles realizada	Completado
Conversión de píxeles indexados a binario realizada	Completado

7. Control de Versiones

Los archivos finales de este sprint fueron subidos al repositorio GitHub, en la rama *main*, bajo la siguiente estructura:

```
None
LAB2/
├── Production/
│     └── SPRINT2/
│          ├── cargar_imagen.m
│          ├── comprimir_paleta.m
│          ├── convertir_binario.m
│          ├── italy.jpg
│          ├── main.m
│          ├── paleta_colores.mat
│          └── serializar_pixeles.m
│     ├── SPRINT3/
│     ├── SPRINT4/
│     ├── SPRINT5/
│     └── SPRINT6/
└── Documentation/
└── src
```

Repositorio: https://github.com/MARTINGONZALEZJOAQUI/LAB2